ROBOTIC MANIPULATION - ELEC-E8126

EXERCISE 5

-Fornaro Gianfilippo 802732-

How to run the grasp planning code

The code is realized in c++ and has a dependency on *boost* library.

The code mainly uses the geometry module of boost geometry

(https://www.boost.org/doc/libs/1_72_0/libs/geometry/doc/html/index.html).

Due to the usage of the double number representation and to the boost library sometimes some intersection points are not detected. In these cases, the code cannot find a solution to the problem and will return an error code. The code can fail the search of a grasping point, in these cases just rerun the code.

The code is delivered as a source code file named “main.cpp”. Along with it, there are the two boost libraries (.lib) the code must be linked with at compilation time. To compile the code you need to have the boost 1.72.0 library distribution somewhere on your environment. You find a trimmed down working copy along with this code in the file “boost_1_72_0.7z” in the repository. Inside there are two folders: a folder containing the two aforementioned .lib libraries and all the boost source code needed for compilation.

Alternatively of building and running the code, you will find an executable called “version2_ex5.exe” that you can simply run to obtain the grasping data file and a visualization of the grasp.

The program takes two input parameters:

[path_to_input_file] [path_to_output_folder]

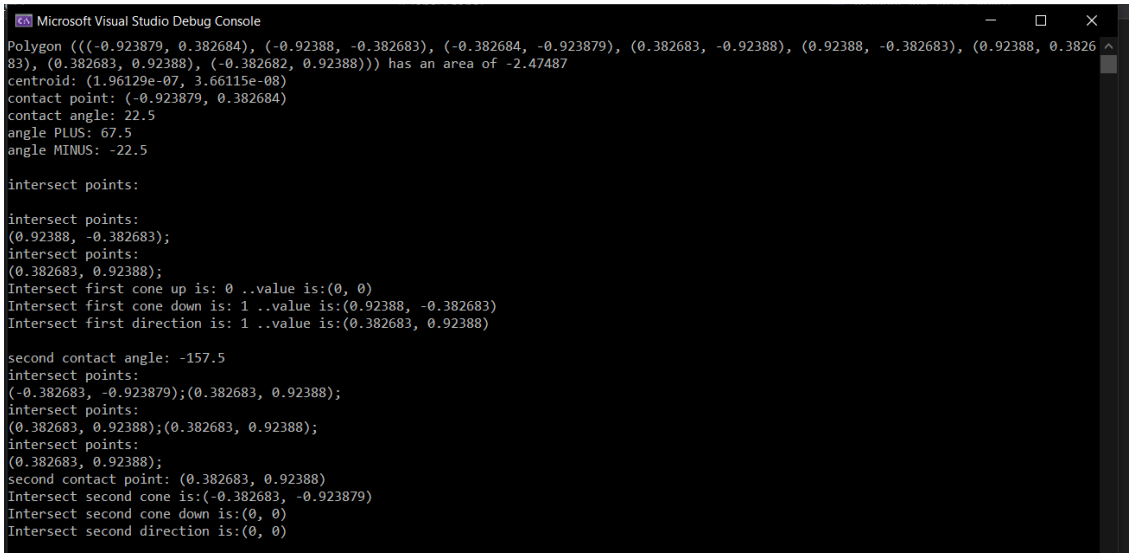
Inside *path_to_output_folder* the program will generate a folder and places the two files in it.

The image file has name “*image_[epoch].svg*”. An example is “*image_1584737700.svg*”.

The grasping data file has name “*grasp_data_[epoch].txt*”. An example is “*grasp_data_1584737700.txt*”

The .exe file has been correctly executed on windows 10 with both PowerShell terminal and visual studio enterprise 2019 ide.

Example of output:



```
Microsoft Visual Studio Debug Console
Polygon (((-0.923879, 0.382684), (-0.92388, -0.382683), (-0.382684, -0.923879), (0.382683, -0.92388), (0.92388, -0.382683), (0.92388, 0.3826
83), (0.382683, 0.92388), (-0.382682, 0.92388))) has an area of -2.47487
centroid: (1.96129e-07, 3.66115e-08)
contact point: (-0.923879, 0.382684)
contact angle: 22.5
angle PLUS: 67.5
angle MINUS: -22.5

intersect points:

intersect points:
(0.92388, -0.382683);
intersect points:
(0.382683, 0.92388);
Intersect first cone up is: 0 ..value is:(0, 0)
Intersect first cone down is: 1 ..value is:(0.92388, -0.382683)
Intersect first direction is: 1 ..value is:(0.382683, 0.92388)

second contact angle: -157.5
intersect points:
(-0.382683, -0.923879);(0.382683, 0.92388);
intersect points:
(0.382683, 0.92388);(0.382683, 0.92388);
intersect points:
(0.382683, 0.92388);
second contact point: (0.382683, 0.92388)
Intersect second cone is:(-0.382683, -0.923879)
Intersect second cone down is:(0, 0)
Intersect second direction is:(0, 0)
```

Example of a run where it was not possible to find a solution:

```
PS C:\Users\gianf\Documents\Docker_shared\ros\src\exercise5\src\solution\version2_ex5\Debug> ./version2_ex5.exe C:\Users
\gianf\Documents\Docker_shared\ros\src\exercise5\data\vertices.txt C:\Users\gianf\Documents\test y
Polygon (((-0.923879, 0.382684), (-0.92388, -0.382683), (-0.382684, -0.923879), (0.382683, -0.92388), (0.92388, -0.38268
3), (0.92388, 0.382683), (0.382683, 0.92388), (-0.382682, 0.92388))) has an area of -2.47487
centroid: (1.96129e-07, 3.66115e-08)
contact point: (0.415155, -0.891408)
contact angle: 135
angle PLUS: 180
angle MINUS: 90

intersect points:
(-0.415155, -0.891408);(0.415155, -0.891408);
intersect points:
(0.415155, -0.891408);(0.415155, 0.891408);
intersect points:
(0.415155, -0.891408);
Intersect first cone up is: 1 ..value is:(-0.415155, -0.891408)
Intersect first cone down is: 1 ..value is:(0.415155, 0.891408)
Intersect first direction is: 0 ..value is:(0, 0)

CANNOT find second contact point (1)! Exit program
```

As you can notice the intersect point of the direction of the force is not detected by the library, therefore the code cannot find a solution. Try to rerun the program.

The grasp planning approach

To select the first grasping point we select two random vertices and between them, we pick up a random point on the edge. This point is the first grasping point. To create the direction of contact we take the angle of the perpendicular line crossing the edge. We compute a grasping angle close to it. This angle will give us the direction of the force approaching the object on the contact point. Each edge is represented by a fixed number of points adjustable with the *definition* "`DELTA_SENSITIVITY`".

Now that we know the angle of contact, we create three *versors*: the direction of contact and the direction of the upside and downside friction cone. The sides of the cone have an angle of 45 degrees, adjustable with the *definition* "`MU_FRICTION_ANGLE`", w.r.t. the direction *versor*. Now we rotate them with an angle equal to the angle of contact and we translate them to the point of contact. The *versor* length is adjustable through the *define* "`VERSOR_LENGTH`".

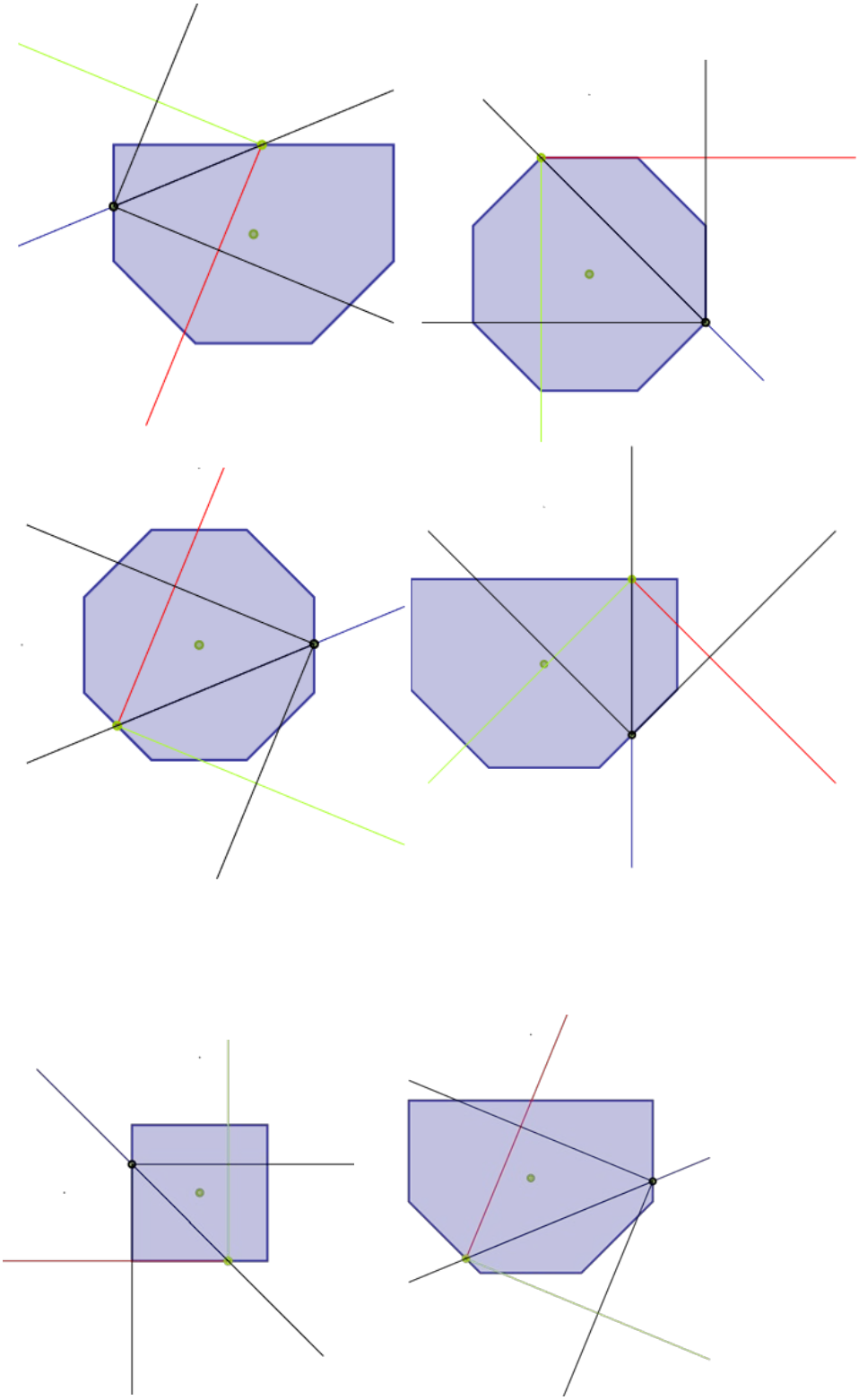
We compute the intercepts of the three *versors* with the polygon. We place the second point of contact at the intersection point between the polygon and the first direction *versor*. As per the second angle of contact, we choose the opposite direction of the first angle of contact. With this procedure, we guarantee that both the points lay inside each other friction cone and therefore grasping stability is guaranteed.

Fornaro Gianfilippo 802732

Visualization of the grasping points, approach direction, and friction cones

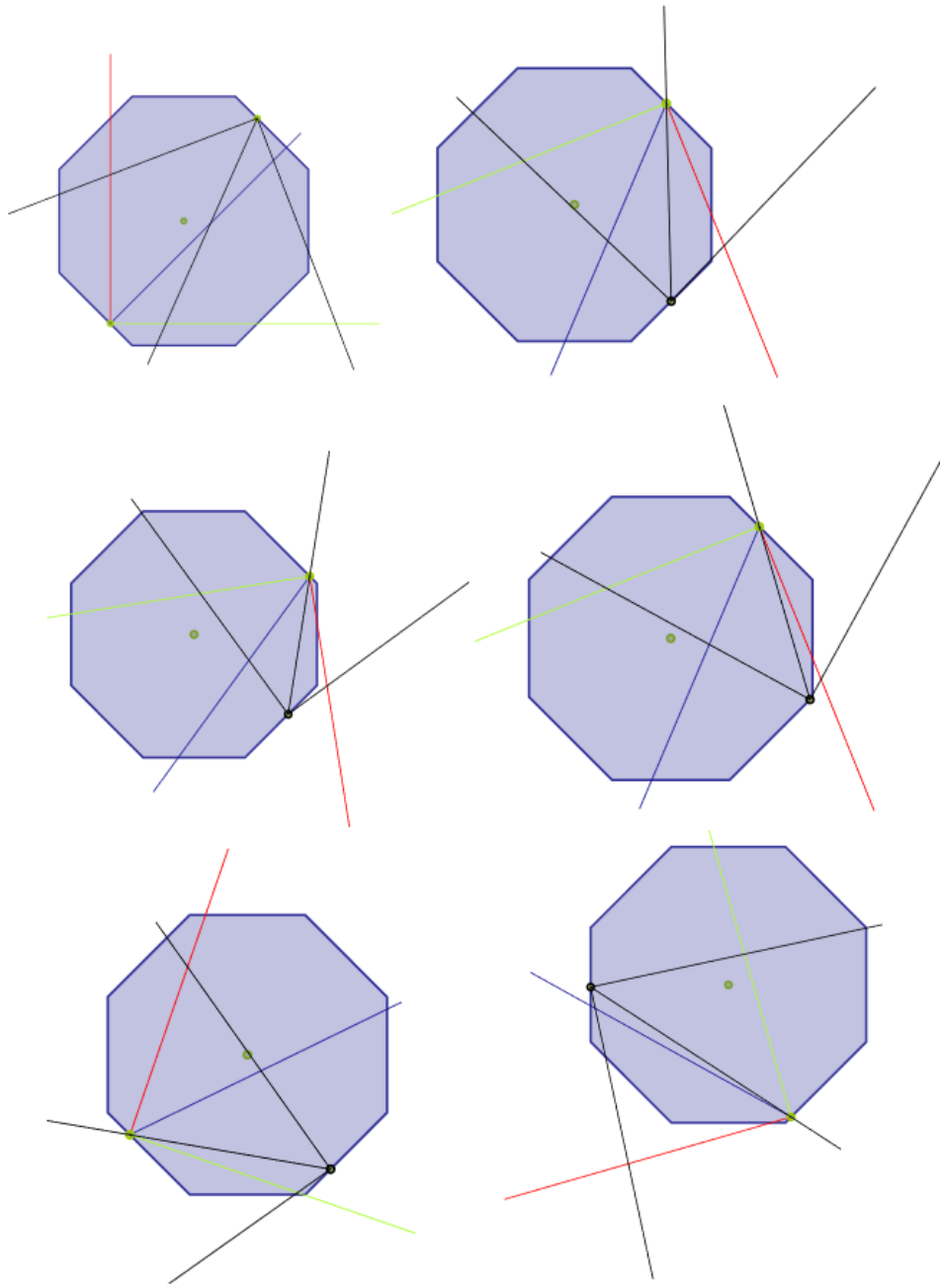
The code produces as output an svg image of the grasping points, the two friction cones and the direction of the force contact. The first point of contact is colored in green, the second point of contact is colored in black. The first friction cone has a side colored in yellow and one in red, while the direction of the force is blue. The second force direction and friction cone are colored in black.

The following images show the result obtained:



Fornaro Gianfilippo 802732

Examples obtain with a more general strategy code (sampling-based grasp). Unfortunately, this code was unstable, and we could not present it.



Fornaro Gianfilippo 802732

Discussion of the results

The two robots go to the two positions indicated by the grasping file and start to stabilize the grasp. Inevitably after some time the forces applied to the object diverge and the two robots will fall to the floor. Due to the different point and precision representation between the output of the program and the one running on Ros, sometimes might happen that the robot does not grasp the object but it remains at a certain small distance from it. In these cases, the simulation does not work properly.

Questions

1)

Is the grasp both force-closure and form-closure, either of, or neither? Explain why.

If we consider the object to be three dimensional, two frictional point contacts are insufficient to yield force closure, as there is no way to generate a moment about the axis joining the two contacts. A force-closure grasp can be obtained with as few as three frictional contacts in a 3d space. So all the forces acting in the 3d space with directions that differ from the plane would produce a moment to which we cannot oppose.

Considering instead a two-dimensional space, given the presence of friction. the two forces are sufficient to guarantee a stable grasp and therefore we have a force-closure grasp if the contact forces are sufficiently high to cancel any external wrench.

When the stationary contacts completely constrain the motion of the body we have a form-closure grasp. In a planar space, we need minimum 3 grasping points to guarantee the form-closure grasp according to the definition $F_i^T V \geq 0$. Therefore our grasp it is not form-closure

2)

If the object was not force-closure, where would we need to place additional contacts to achieve it?

We would have had to trace the friction cone from the first point of contact and place the second contact inside this cone. Now we should have checked that the first point laid inside the friction cone of the second point.

The same concept can be applied to higher dimensions where we will have a hyper-cone instead of a cone.

3)

For the bonus task specifically report the approach and the metric used for planning the stable grasp;

As for the optimal strategy, it is not implemented in this code but we report a simple strategy to obtain it. Knowing the intersection points with the polygon, we take apart all the points that lay between the intersections or in case of only one intersection (only one side of the cone crosses the polygon) from the intersection point to the point of contact. From this collection of points, we pick up the point with the minimum distance from the centroid of the polygon. This point will be the second grasping point. The second contact angle will be the angle conjoining the two points. In this case, optimality is guaranteed.

Unfortunately, due to library errors such as missing to recognize intersection points, it was unfeasible to realize the optimal algorithm. Perhaps, by using *big numbers* instead of *double* to represent the points, we can achieve a better result, avoiding this erroneous behavior.

An estimate of time spent on this exercise

28 hours.